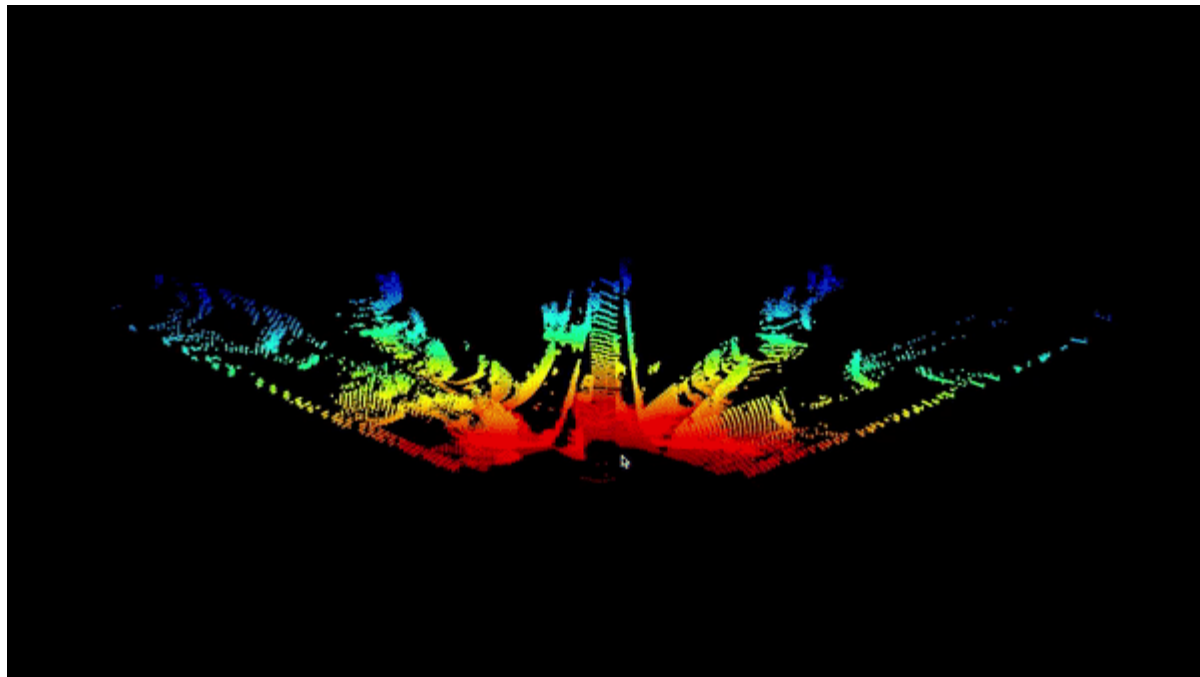


Autonomous Systems and Mobile Robots

Exteroception - LiDAR

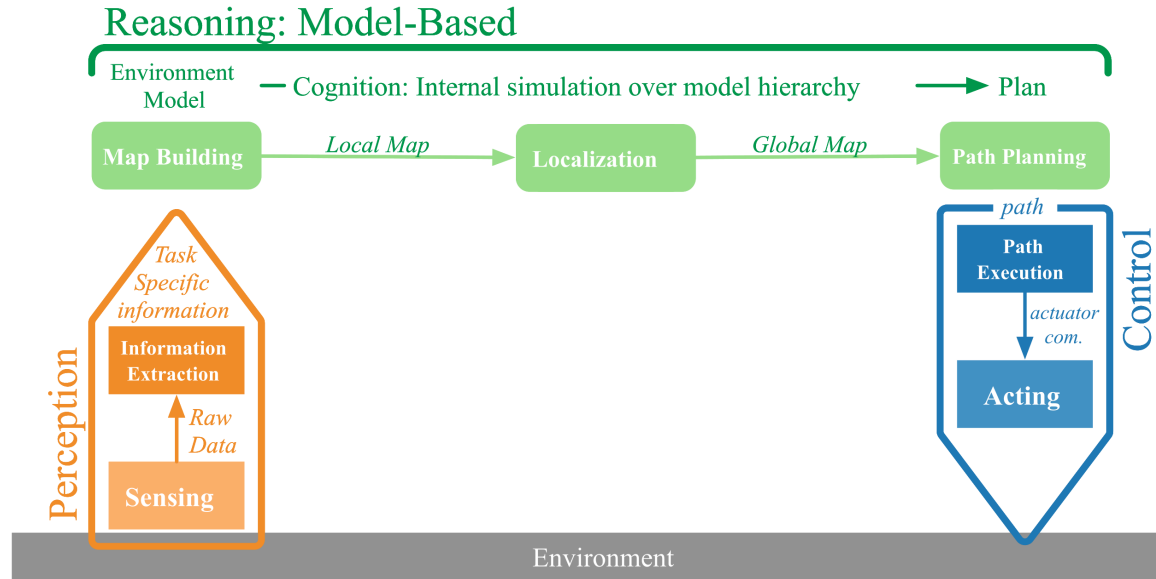


Mallick, *LearnOpenCV* 2023 — learnopencv.com



Janosch Bajorath
j.bajorath@uni-muenster.de

Recap



Sensor — LiDAR

How a 2D LiDAR works, what it measures, and how to integrate it into the robot: URDF, Gazebo plugin, ROS 2 bridge.

Information Extraction

How to turn raw distance arrays into useful information: indexing by bearing, coordinate transforms, comparing against odometry.

Semester at a Glance

#	Date	Theme
0	14.04	Python Intro
1	21.04	ROS2 Packaging + PID Control
2	05.05	Robot Body and Blind Locomotion
3	12.05	Exteroception — LiDAR ← you are here
4	19.05	MiniLab 1 Briefing
—	26.05	Pfingsten — no session
5	02.06	Troubleshooting
6	09.06	FK/IK Robotic Manipulator
7	16.06	MiniLab 2 Briefing
8	23.06	Troubleshooting
9	30.06	Recap

Today's Content

Sensor — LiDAR

Proprioception vs exteroception

Active sensor: emission–return model

Working principle: rotating laser, ring of distances

Key parameters: angular resolution, range, scan rate

Information Extraction

`sensor_msgs/LaserScan` — message fields

Forward-index formula: bearing → array index

Systematic vs stochastic sensor error

TF2: querying transforms in Python

Sensing the World

From proprioception to exteroception

Sensing Direction

Proprioceptive sensors measure the robot's own internal state.

- No signal emitted, no environment sampled
- Example: wheel encoder → odometry
- Tells the robot *where it thinks it is*

Covered in Week 2: odometry via `diff_drive` plugin

Exteroceptive sensors measure the environment around the robot.

- Emit or receive signals from the world
- Example: LiDAR → distance ring
- Tells the robot *what is in front of it*

Today: add LiDAR to the Week 2 robot

Signal Origin

A sensor is **active** if it emits its own signal and measures the return.

A sensor is **passive** if it only receives ambient signals.

	Emits	Measures
LiDAR	Laser pulses	Return time → distance
Camera	—	Reflected ambient light

The emission–return model makes LiDAR independent of ambient lighting conditions.



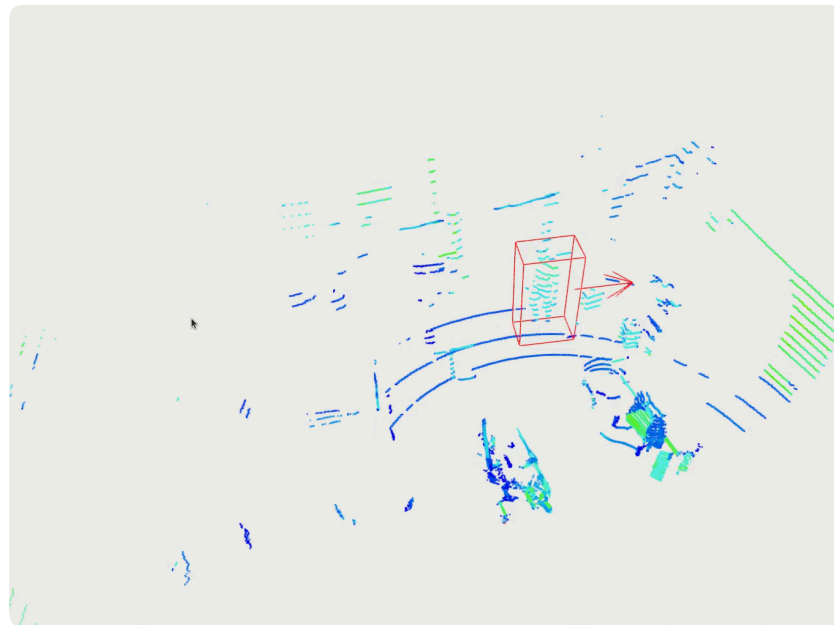
DFKI field robot — dfki.de

Sensor modalities

Raw sensor data — RGB channels (left) vs. point cloud (right).



Camera



LiDAR

LiDAR

Light Detection and Ranging

Working principle of a 2D Lidar

A rotating laser emits pulses in a plane. For each angle the sensor records the distance to the nearest surface.

The result is a **ring of distances** — one value per angle step.

Key parameters:

Parameter	Meaning
Angular resolution	Angle between consecutive beams
Max range	Furthest measurable distance
Scan rate	Full rotations per second (Hz)



Barrile et al., *Sensors* 2023, 23, 2534 — [doi:10.3390/s23052534](https://doi.org/10.3390/s23052534)

The LaserScan Message

Data types, message format, and indexing

Laser Scan Data Structure Types

2D structured

Single rotating plane · fixed angular grid

`sensor_msgs/LaserScan`

← [this course](#)

2D unstructured

Irregular point set in a plane

No standard ROS2 type

Rare in practice

3D structured

Organised grid (e.g. depth camera)

`sensor_msgs/PointCloud2`

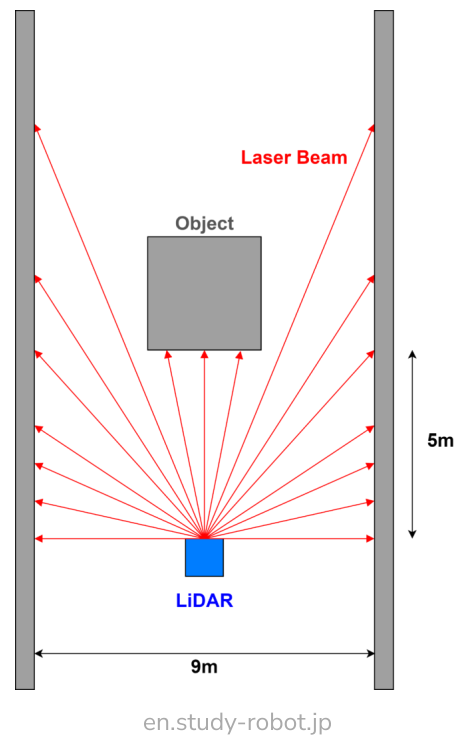
3D unstructured

Arbitrary point cloud (e.g. multi-beam LiDAR)

`sensor_msgs/PointCloud2`

sensor_msgs/LaserScan

Field	Meaning
<code>ranges[]</code>	Distance per angle step (metres)
<code>angle_min</code>	Angle of the first beam (rad)
<code>angle_max</code>	Angle of the last beam (rad)
<code>angle_increment</code>	Angular step between beams (rad)
<code>range_min/max</code>	Minimum/Maximum valid distance



Extracting Information from the LaserScan Message

General formula:

$$i = \text{round}\left(\frac{\theta - \text{angle_min}}{\text{angle_increment}}\right)$$

Worked example — sensor as configured (360 samples, $-\pi$ to π):

Target bearing	Calculation	Index
Forward ($\theta = 0$ rad)	$(0 - (-\pi)) / (\pi/180)$	180

Edge case: if `ranges[i]` is `inf`, the beam returned no echo — the surface is beyond `range_max`, or the beam was absorbed.

Coordinate Transforms

From sensor frame to robot frame

tf2 package - Solving the Transform Math

Transforming a Scan Point from `lidar_link` to `base_link`

Three steps to move a point between frames:

1. **Get the transform** — ask TF2 for the relationship between the two frames at the latest time.
2. **Build the point** — express the obstacle in the source frame using polar → Cartesian.
3. **Apply the transform** — project into the target frame.

```
import tf2_geometry_msgs must be present even if never
called directly — it registers the PointStamped handler at
import time.
```

```
import math, rclpy
import tf2_geometry_msgs
from geometry_msgs.msg import PointStamped

# 1. Get transform (latest available)
transform = tf_buffer.lookup_transform(
    'base_link',          # target frame
    'lidar_link',         # source frame
    rclpy.time.Time()    # latest available
)

# 2. Build point in lidar_link frame
pt = PointStamped()
pt.header.frame_id = 'lidar_link'
pt.point.x = r * math.cos(theta)
pt.point.y = r * math.sin(theta)
pt.point.z = 0.0

# 3. Apply transform
result = tf2_geometry_msgs.do_transform_point(
    pt, transform
)
```

Sources of Error

Why sensor measurements deviate from ground truth

Measurements deviate from ground truth

Exteroceptive sensors observe physical phenomena. Real observations carry error.

Systematic error

Consistent bias — the measurement is always wrong in the same direction.

Reproducible and potentially correctable by calibration.

Stochastic error

Random noise — the measurement varies unpredictably around the true value.

Cannot be removed by calibration; reduced by averaging or filtering.

Simulation vs. Reality

Gazebo (this course)

- Sensor sits at **exactly** the URDF-configured origin
- Range measurements are **noise-free** — every value is ground truth
- All surfaces return valid echoes — no absorption, no reflections
- Deterministic: same world → same scan, every time

"The model is the world."

Real LiDAR

- Calibration bias — offset in `angle_min` or scale in `ranges` or `mounting origin`
- Specular / dark surfaces — glass and black materials absorb or scatter beams
- Shot noise — photon-level randomness in time-of-flight
- Motion distortion — fast robot + slow scan rate → stale data mid-rotation

"The measurement approximates the world."

What You Will Build

Your robot today

Exercise Tasks

Sensor integration

hardware

Attach the LiDAR to the robot model · connect it to the simulation · publish
`/scan`

Signal processing

measurement

Index the scan by bearing · extract forward distance · compare against
odometry

Coordinate transforms

geometry

Query TF2 · convert polar to Cartesian · express a scan point in `base_link`

LiDAR on the diff-drive robot

